

Basic-Concepts-Java(Day-1)

1. Machine-Level Language

Machine language is the only language a computer's CPU understands directly.

- It consists of **0s and 1s (binary)**.
- Every CPU instruction is represented as binary.
- It is hardware-specific.

Example:

```
10110000 01100001
```

The CPU reads these bits directly and executes them.

Advantages

- Fastest execution
- No translation required

Disadvantages

- Extremely difficult for humans to read and write
 - One small mistake can cause errors
 - Different processors use different machine languages
-

2. High-Level Language

High-level languages are designed for humans.

Examples:

- Java
- Python

- C++
- JavaScript
- C#

Example Java program:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

This is much easier to understand than binary.

Advantages

- Easy to learn
- Easy to debug
- Faster development
- Portable

Disadvantages

- Cannot be executed directly by the CPU
- Needs translation into machine language

How High-Level Code Executes

```
Programmer  
↓  
High-Level Code  
↓  
Compiler / Interpreter  
↓  
Machine Code (0s and 1s)  
↓  
CPU executes it
```

Compiler vs Interpreter

Both translate high-level code into something the computer understands.

Compiler

A compiler translates the **entire program at once**.

```
Source Code
  ↓
Compiler
  ↓
Machine Code
  ↓
Run
```

Example languages:

- C
- C++

Example:

```
printf("Hello");
```

After compilation:

```
hello.exe
```

Now the CPU executes the executable directly.

Advantages

- Very fast execution
- Errors are found before running

Disadvantages

- Compilation takes time
- Platform dependent

Interpreter

An interpreter translates **one line at a time while the program runs**.

```
Source Code
  ↓
Interpreter
  ↓
Execute line by line
```

Example languages:

- Python
- JavaScript

Example:

```
print("Hello")
```

Interpreter reads:

```
Read line
  ↓
Translate
  ↓
Execute
  ↓
Next line
```

Advantages

- Easy to test
- No separate compilation

Disadvantages

- Slower execution
- Translation happens every run

Compiler vs Interpreter Comparison

Feature	Compiler	Interpreter
Translation	Entire program	One line at a time
Speed	Faster	Slower
Error reporting	After compilation	During execution
Output	Executable file	No executable
Execution	Runs after compilation	Runs immediately
Examples	C, C++	Python, JavaScript

Where Does Java Fit?

Java uses **both a compiler and an interpreter**.

This is what makes Java special.

Java Compilation Process

Suppose you write:

```
System.out.println("Hello");
```

Step 1

You save:

```
Main.java
```

Step 2

Java Compiler (`javac`)

```
Main.java  
  ↓  
javac  
  ↓  
Main.class
```

Notice:

It **does not produce machine code**.

Instead, it produces **bytecode**.

What is Java Bytecode?

Bytecode is an intermediate language.

It is **not**:

- Java source code
- Machine code

It is a special instruction set designed for the JVM.

Example:

```
aload_0
getstatic
invokevirtual
return
```

Humans rarely write bytecode directly.

What is the JVM?

JVM stands for **Java Virtual Machine**.

It is software that acts like a virtual computer.

```
Java Program
↓
Bytecode (.class)
↓
JVM
↓
Machine Code
```

↓

CPU

The JVM understands bytecode.

The CPU understands machine code.

So the JVM converts bytecode into machine code.

Why Does Java Need the JVM?

Different operating systems use different machine code.

For example:

Windows CPU:

Machine Code A

Linux CPU:

Machine Code B

Mac CPU:

Machine Code C

If Java generated machine code directly, you'd need a different compiled program for each operating system.

Instead:

Java Code

↓

Bytecode

↓

JVM (Windows)

↓

Windows Machine Code

or

Java Code

↓

Bytecode

↓

JVM (Linux)

↓

Linux Machine Code

or

Java Code

↓

Bytecode

↓

JVM (Mac)

↓

Mac Machine Code

The **same** `.class` **file** works everywhere because each platform has its own JVM.

This is why Java is called **platform independent**.

| **Write Once, Run Anywhere (WORA)**

Does JVM Interpret or Compile?

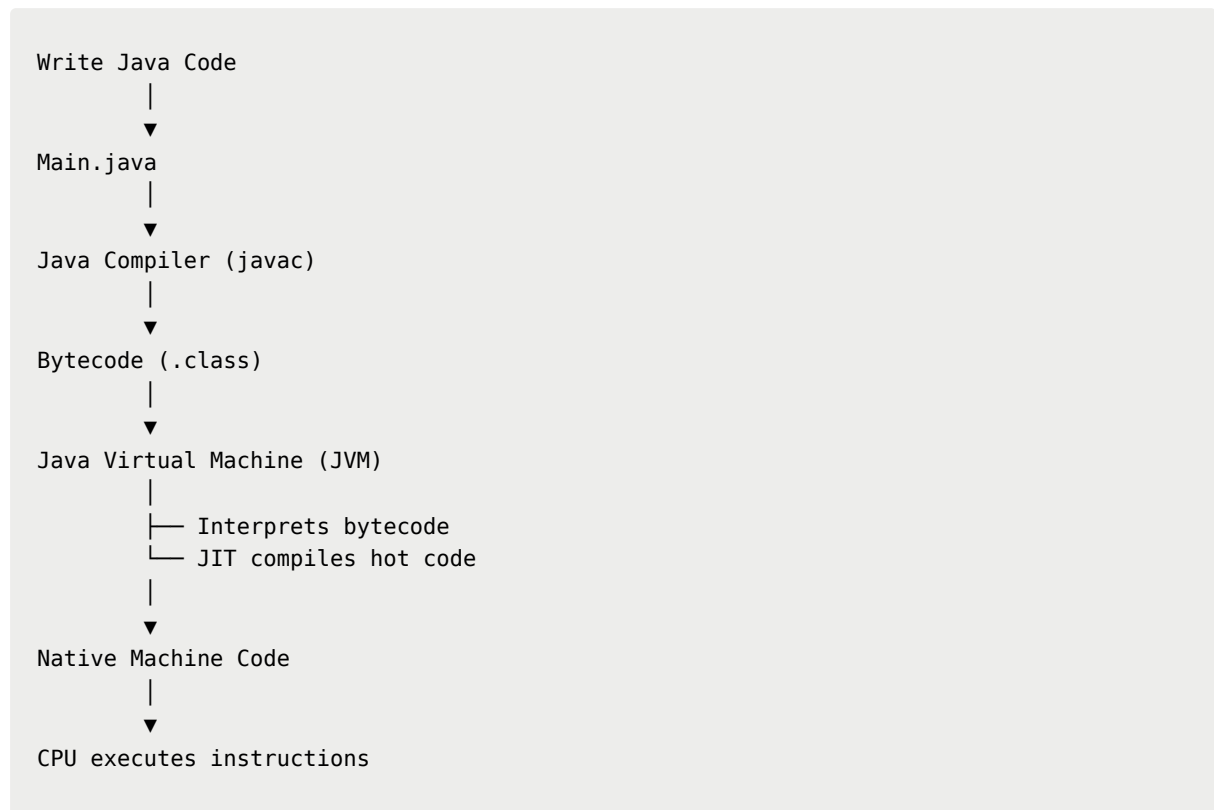
Originally, the JVM interpreted bytecode.

Modern JVMs use **JIT (Just-In-Time) compilation**:

1. Start by interpreting bytecode.
2. Detect frequently executed ("hot") code.
3. Compile that code into native machine code.
4. Reuse the compiled version for better performance.

This makes Java much faster than pure interpretation while still remaining portable.

Complete Java Execution Flow



Why Java Uses Both Compilation and Interpretation

- **Compilation** (`javac`) converts Java source code into platform-independent bytecode.

- **Interpretation/JIT** (inside the JVM) converts bytecode into machine code for the current operating system.
- This combination gives Java:
 - **Platform independence** (same bytecode runs anywhere with a JVM)
 - **Good performance** (thanks to JIT compilation)
 - **Security** (the JVM verifies bytecode before execution)
 - **Portability** (no need to recompile for every operating system)

Summary

Concept	Meaning
Machine Language	Binary instructions (0s and 1s) executed directly by the CPU
High-Level Language	Human-readable programming language (e.g., Java, Python)
Compiler	Translates the entire program into machine code (or bytecode in Java) before execution
Interpreter	Translates and executes code line by line during execution
Bytecode	Platform-independent intermediate code produced by the Java compiler (<code>.class</code> files)
JVM	Java Virtual Machine that executes bytecode by interpreting it and using JIT compilation
Java's Approach	Compiles source code to bytecode, then the JVM interprets/JIT-compiles it into native machine code
Main Benefit	Write Once, Run Anywhere (WORA) due to platform-independent bytecode and platform-specific JVMs